Quantile Spine

A small, fast way to track millions of value distributions at once

A design white paper for the **monatq** project · design, reasoning, and first head-to-head measurements

The one-minute version. When studying a neural network, we often want to know, for *every single position* inside a tensor, what values tend to appear there: what is typical, what is rare, how extreme do things get? A tensor can have millions of positions, and each one sees a long stream of values — far too much to store. The current tool for this, the **t-digest**, works well but needs roughly **5 kilobytes of memory per position** and does complicated bookkeeping on every update.

Quantile Spine is a redesign built on a simple observation: we never look at the bookkeeping — we only ever ask for the answers. So it stores the answers directly: a fixed set of **percentile marks** (the "spine"), an exact list of the record-lowest and record-highest values, and exact counters for special values such as zero. That is about **370 bytes per position — roughly 13× smaller** — and every position is updated with the same simple arithmetic, which modern CPUs can do for many positions simultaneously. The sketch also **watches its own surprise**: every incoming batch is compared against what the spine expected, and disagreement makes the sketch adapt faster — so drifting or suddenly-changing streams are tracked instead of averaged away. A first implementation confirms the design (Section 9): the predicted **13×** memory saving exactly, updates **1.2–1.9×** and queries **5× faster** than the t-digest, lower error — mean *and* worst case — on every distribution tested, and spike- and drift-handling better by orders of magnitude — with extremes stored exactly.

1. The problem: millions of tiny streams

A neural network processes data as **tensors** — large grids of numbers. Run the network on one input and every position in the grid takes some value. Run it on ten thousand inputs and every position has seen ten thousand values. Those values form a little stream of history, one stream per position (Figure 1).

The **monatq** library exists to answer questions about those streams:

- *"What is the typical value at this position?"* (the median)
- *"What range covers 98% of what this position sees?"* (the 1st to 99th percentile)
- *"How extreme does it get?"* (the record highs and lows)

These questions drive real decisions — for example, choosing how aggressively a model can be compressed (quantization), or understanding which neurons behave unusually (interpretability).

The catch is scale. Storing every value at every position would take *number of samples × number of positions* numbers — easily hundreds of gigabytes. So each position must keep only a **tiny summary** of its stream, one that can still answer percentile questions afterwards.

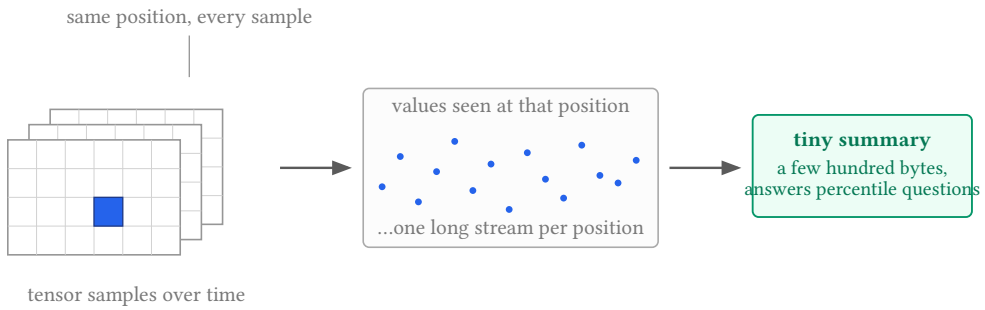


Figure 1: Every position in the tensor sees its own stream of values across samples. We cannot store the streams — each position keeps a small summary instead.

2. Percentiles, in plain words

Line up everything a position has ever seen, smallest to largest. The **median** (50th percentile) is the value halfway along the line: half of all observations fall below it. The **95th percentile** is the value with 95% of observations below it. Percentiles describe a distribution in the way people actually use it: *typical value, normal range, extremes* (Figure 2).

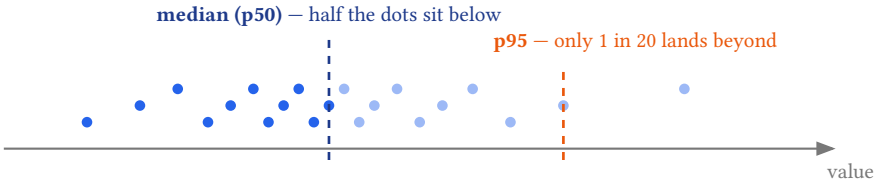


Figure 2: Percentiles read a stream of values the way people do: "what is typical, and what is rare?" Dark dots are below the median.

The technical term for a percentile mark is a **quantile** — "the 0.95 quantile" means the same as "the 95th percentile". We use the friendlier word from here on.

2.1. How it is done today: one t-digest per position

The classic tool for summarising a stream this way is the **t-digest** [1]. It compresses the stream into a few hundred little clusters, keeping clusters small near the extremes (where precision matters most) and letting them grow in the middle. It is a genuinely good algorithm, and it is what monatq uses today — one t-digest per tensor position.

But per position it is expensive, and the expense is multiplied millions of times:

- **Memory.** At the standard accuracy setting, each position reserves room for 610 clusters — about **4.9 KB**. One million positions ≈ 5 GB just for summaries.
- **Work per update.** Deciding which clusters to merge is a chain of data-dependent decisions. Every position takes its own path, so the CPU cannot process positions in lockstep — a large hidden cost at tensor scale.
- **No guarantee.** Perhaps surprisingly, the t-digest's accuracy is empirical: there is no mathematical bound on how wrong it can be, and adversarial inputs that fool it are known.

The redesign below keeps what the t-digest gets right — precision focused on the extremes — and removes the machinery.

3. The idea: store the answers, not the machinery

Here is the whole idea in one sentence:

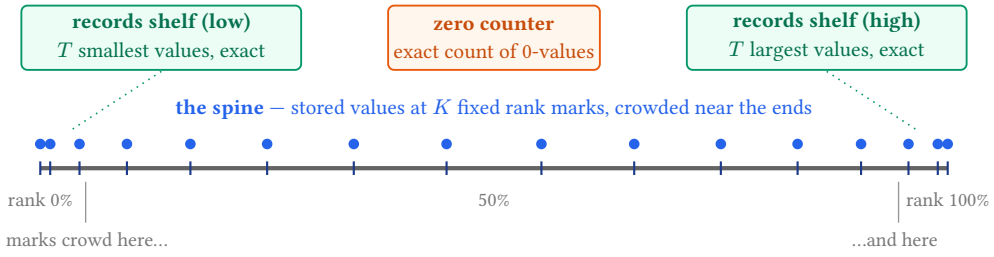
> *Since we only ever ask for quantiles, store the quantiles themselves — a fixed set of marks — and keep them up to date as data streams in.*

Each position keeps a **spine**: the values at K fixed rank positions (for example $K = 64$ marks: "the value 0.1% of the way up the sorted stream, the value 0.5% of the way up, … the value 99.9% of the way up"). The marks are placed **densely near the two ends** and sparsely in the middle — exactly where precision is needed, the same insight that makes the t-digest good (Figure 3).

Three small companions make the summary complete:

- **A records shelf.** The T smallest and T largest values ever seen, kept **exactly** (say $T = 8$ each), plus a short *recent-window* shelf that restarts whenever the distribution visibly changes — so records can be dated: *all-time* versus *since things last changed*. Record extremes are the one thing approximation should never touch — so they are not approximated at all.
- **A zero counter.** Neural networks produce huge numbers of *exact zeros* (a ReLU activation outputs zero about half the time). A smooth curve cannot represent a giant spike at one value, but a simple counter can — exactly.

- **A surprise gauge.** One small number tracking how much recent batches have disagreed with what the spine expected. It is computed for free inside the update (Section 6) and is what lets the sketch *adapt* when the stream changes instead of averaging the change away. Cost: 4 bytes.



Total per position: $K = 64$ anchors + records + counters + surprise gauge $\longrightarrow$ **≈ 370 bytes** (t-digest: ≈ 4 900 bytes)

Figure 3: Everything one position stores. The spine holds values at fixed rank marks, dense near the extremes; the records shelves keep the true extremes exactly; the zero counter handles the ReLU spike. A few more bytes (not drawn) hold the windowed records, a promoted-spike counter, and the surprise gauge of Section 6.

One more saving is almost free. Every position sees *the same number* of samples — the stream lengths are always identical, because each incoming tensor contributes exactly one value to every position. So the sample count, and all bookkeeping derived from it, is stored **once for the whole tensor** rather than once per position. The t-digest cannot exploit this; its per-position cluster weights must each carry their own counts.

4. How it learns: blend, then re-read

New tensors arrive one at a time, but the spine does not update on every single value. Instead, values are collected into a **batch** (a few hundred samples), and the batch is folded in all at once. Batching is what monatq already does for the t-digest — but for the spine the folding step becomes strikingly simple:

1. **Sort** the batch for this position (its few hundred values).
2. **Update the records.** Compare the batch's smallest/largest values with the shelves; keep the best T of each. Exact, and trivially cheap.
3. **Blend and re-read.** The spine describes the distribution of the N values seen so far; the sorted batch describes the B new ones *exactly*. Mix the two in proportion — N parts old, B parts new — and read off the values at the K fixed rank marks again. Those become the new spine (Figure 4).
4. **Note the surprise.** While sweeping, compare where the batch's values landed against where the spine expected them. The largest gap — the position's *surprise* — costs one extra comparison per value and controls how the mixing weights are set (Section 6): a stream behaving as usual blends gently; a stream that has visibly changed is absorbed quickly.

The blend is a weighted average of two curves, computed in a single sweep. There are no clusters to manage, no decisions to make — every position performs the same fixed arithmetic. This is what makes the update fast, and (as Section 7 explains) lets the CPU process eight positions at once.

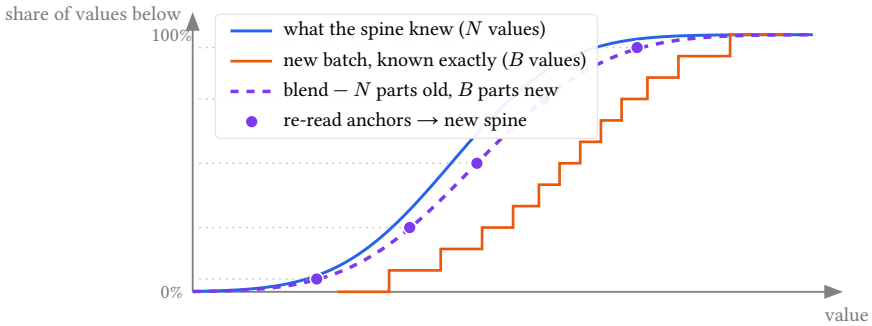


Figure 4: One update step. The rising curves show, for each value on the horizontal axis, what share of observations falls below it. The old knowledge (blue) and the exactly-known new batch (orange) are blended in proportion; the spine is then re-read at its fixed rank marks (purple dots).

Deeper dive — the update rule, formally

Let $\hat{F}_{\text{old}}$ be the distribution implied by the spine (interpolated through the anchors), $\hat{F}_B$ the exact empirical distribution of the sorted batch, N the running sample count and B the batch size. The merged distribution is the mixture

$$\hat{F}_{\text{new}} = \lambda \hat{F}_{\text{old}} + (1-\lambda)\hat{F}_B, \quad \lambda = \frac{N_{\text{eff}}}{N_{\text{eff}} + B},$$

where $N_{\text{eff}} = \min(N, N_{\max})$ caps the weight of history (fading memory) and λ is further reduced when the surprise gauge fires — both defined in Section 6. The new anchors are the mixture's quantiles at the fixed ranks: $a'_j = \hat{F}^{-1}_{\text{new}}(q_j)$. Because both inputs are monotone and sorted, this inversion is a single coordinated linear sweep costing $O(K+B)$ — no data-dependent branching. The only information lost anywhere in the pipeline is the interpolation between anchors of $\hat{F}_{\text{old}}$; every other quantity in the rule is exact. The entire error analysis therefore reduces to controlling that one interpolation — which the refinements below make small.

5. Why it stays accurate: four refinements

Reading a curve at fixed marks loses a little information each time. Done naïvely, thousands of updates could let those little losses pile up. Four refinements — each simple on its own — keep the total error tiny. Together they are what turn the sketch from "plausible" into "sound".

5.1. Wiggle the ruler (so errors cancel instead of piling up)

Each blend step commits a tiny reading error at each mark. The danger is not the size of one error — it is that the *same* error, repeated over thousands of updates in the same direction, adds up in a straight line.

The fix is old statistical wisdom: **add a tiny random wiggle**. At each update, the rank marks are shifted by a small random amount (well under the gap between marks) before reading, then treated as the canonical marks. The reading errors now point in a random direction each time — sometimes a hair high, sometimes a hair low — and cancel out. After M updates the accumulated error grows like $\sqrt{M}$ instead of M: after 10 000 updates, that is the difference between **100× smaller error** and no improvement at all. The same trick, applied to a different sketch, is what gave the well-known KLL algorithm [2] its mathematical guarantee.

A second effect helps quietly: early errors fade. An error committed when the sketch had seen m batches is carried forward with weight proportional to m/M — mistakes made when the sketch was young are mostly washed away by later data.

5.2. Use bell-curve graph paper (so the lines are nearly straight)

Between two marks, the spine has to guess by drawing a connecting line. How much that guess loses depends entirely on how *curved* the truth is between the marks. So: change the paper.

Plotted on ordinary axes, the quantile curve of bell-shaped data bends hard at both ends — exactly where straight-line guesses fail. But plotted against **bell-curve-spaced axes** (the statistician's "probit scale"), the same curve becomes a **perfectly straight line** whenever the data is bell-shaped (Figure 5) — and neural-network tensors are usually close to bell-shaped. A straight line between marks loses *nothing*; a nearly-straight one loses almost nothing. The trick costs no memory: the axis spacing is a fixed table shared by all million positions.

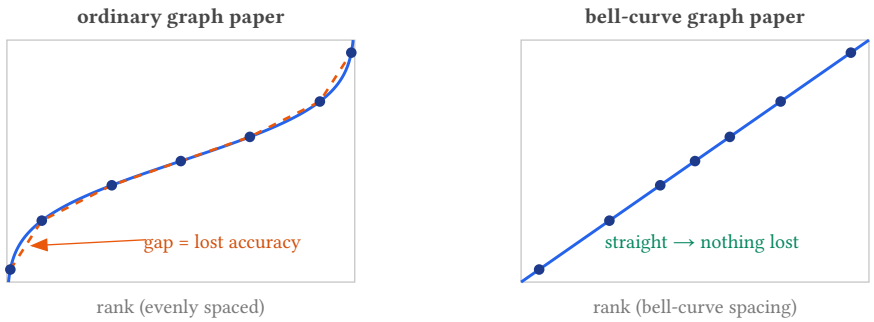


Figure 5: The same bell-shaped distribution drawn twice. Left: on ordinary axes the quantile curve (blue) bends at the ends, and straight lines between anchors (orange, dashed) miss it. Right: on bell-curve-spaced axes the curve *is* a straight line — connecting the same anchors is exact.

For data that is bell-shaped after taking logarithms (also common in neural networks), the same idea applies with a log transform first. Where extra smoothness is wanted, the straight connecting lines can be upgraded to gentle curves (monotone cubics [3]) — a standard, safe interpolation that never invents spurious wiggles.

5.3. Count the zeros separately

About half of all values after a ReLU activation are *exactly zero* — a giant spike at a single point (Figure 6). Any method built on smooth curves — the t-digest included — smears such a spike out. The spine sidesteps the problem: a per-position counter records the zeros exactly, and the spine models only the nonzero values, which are smooth and well-behaved. At query time the two parts are recombined exactly. Cost: 4 bytes. (Spikes at *other*, unanticipated values are caught at run time and promoted to counters of their own — see Section 6.)

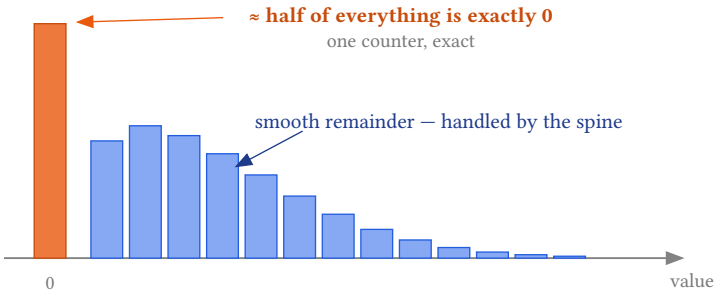


Figure 6: Values at a position after a ReLU activation. A smooth curve cannot honestly represent the spike at zero; a counter represents it perfectly.

5.4. Be exact whenever exactness is free

Two moments in a sketch's life allow exactness at zero cost, and the spine takes both:

- **Early life.** Until a position has seen more values than it has anchors ($N \leq K$), the sorted values simply *are* the state — nothing is approximated. Blending only begins once the stream outgrows the storage.
- **The extremes, always.** The records shelves keep the T true smallest and largest values at all times. Questions about the far tail — "the worst value in a hundred thousand" — are answered from the shelf, *exactly*. The t-digest, by contrast, always interpolates near the extremes. For the deep region between the shelf and the outermost anchor, a classical extreme-value formula [4] can be fitted *at query time*, using no stored memory at all: queries are rare, so they can afford to think.

> **Deeper dive — the accuracy picture in numbers**
>
> Let Δ be the gap between neighbouring rank marks (on the transformed axis) and $M = N/B$ the number of blend steps. One blend's reading error at a mark is $O(\Delta^2)$ for straight-line interpolation — and vanishes entirely for bell-shaped data on the probit axis; monotone cubics reduce the generic case to $O(\Delta^4)$. With random dithering the errors form a zero-mean sequence with fading influence (m/M for the error from blend m), so the accumulated error concentrates at $O\left(\Delta^2\sqrt{M/3}\right)$ (Azuma), rather than the $O(\Delta^2 M)$ of the naive scheme. (Fading memory, Section 6, additionally caps M at $N_{\max}/B$, making

the bound uniform over arbitrarily long streams.) Balancing interpolation error against the unavoidable statistical noise of N samples (quantile CLT: standard error $\propto 1/\sqrt{N}$) shows there is no benefit in growing K beyond $\Theta(N^{1/4})$ — for $N = 10^6$, a spine of $K \approx 64$ marks already sits at the statistical noise floor. That is the formal justification for "64 numbers are enough".

6. How it adapts: surprise as a signal

The refinements above assume the stream keeps behaving like itself. Real streams are not always so polite: a distribution can drift as data changes, a regime can flip outright (a new dataset, a model edit), and a spike can hide at a value nobody anticipated. Rather than hoping such surprises never happen, the spine *measures* them — and the measurement turns out to be nearly free.

6.1. A surprise gauge, free of charge

The blend step already walks the sorted batch against the spine's belief. One extra comparison per value records the largest disagreement between where the batch's values landed and where the spine expected them — a single number D per position, the position's **surprise**. Statistics says exactly how large D should be when nothing has changed (it shrinks like one over the square root of the batch size), so "surprising" has a principled threshold rather than a tuned knob. The gauge is 4 bytes of state; the extra work hides inside the sweep the spine was doing anyway. It is also a useful output in its own right: a per-position **surprise map** shows exactly which neurons changed behaviour.

6.2. Memory that fades, a gain that reacts

The plain blend rule — N parts old, B parts new — never forgets. After a million samples, a batch of five hundred moves the spine by less than a twentieth of a percent, so a drifting stream is chased with ever-growing lag. Two changes fix this:

- **Cap the memory.** Blend as if the spine had seen at most $N_{\max}$ values. Old evidence then fades geometrically, with an explicit, chosen time constant — and the accuracy guarantee survives; in fact it becomes *uniform*: the error bound no longer grows with stream length at all.
- **Let surprise open the gate.** When the gauge fires, the blend tilts further toward the new batch — the more surprising, the stronger the tilt, in the spirit of a Kalman gain. A genuinely shifted stream rewrites the spine within a few batches instead of thousands (Figure 7); ordinary noise, below the threshold, changes nothing at all.

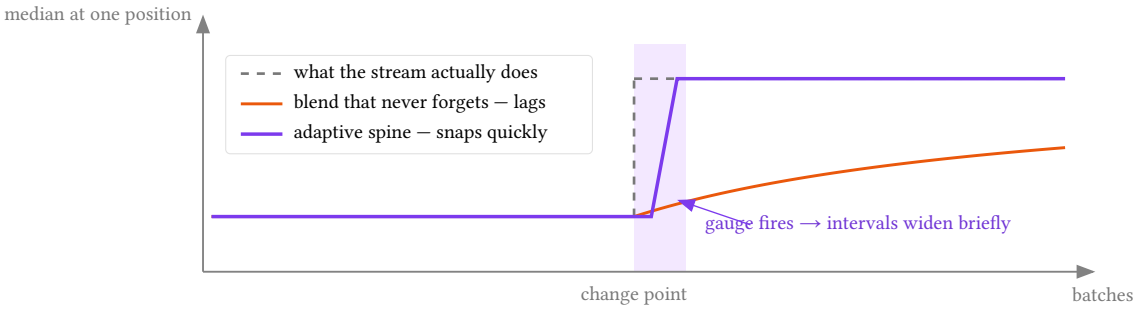


Figure 7: A regime change, seen by one position. The never-forgetting blend (orange) closes the gap only in proportion to elapsed stream length — fifty batches later it is still far off. The adaptive spine (purple) notices the change on its surprise gauge, restarts, and snaps to the new behaviour within a few batches, reporting widened intervals during the brief lag (shaded).

6.3. When the world truly changes: restart, and a second shelf

Sustained surprise — several loud batches in a row — means the old distribution is simply gone. A tiny two-bit state machine per position (*calm* → *alert* → *restart*) makes the response explicit. On restart, the spine keeps its anchors as a warm starting guess but drops their weight, so the next few batches dominate. The restart is expressed entirely through the state bits, so the sample count stays a single global number — the shared-stream saving of the base design is preserved.

Restarts also solve a quiet problem with records: an all-time record is forever, and after a regime change it describes a distribution that no longer exists. This is what the **windowed shelf** is for — the extremes *since the last change-point*, cleared at each restart. Queries can answer both "worst ever" and "worst lately", and say which is which.

6.4. Hidden spikes become counters

The zero counter handles the one spike we can predict. But spikes appear at other values too: saturation limits, clamped activations, a dead neuron stuck at a constant. The sorted batch betrays them immediately — a spike is a run of *tied values*, and ties are counted during the sweep at no extra cost. A value that keeps showing up heavily is **promoted**: it gets its own exact counter, just like zero, and the spine models only the smooth remainder. The worst case for the interpolation assumption is thereby converted into the best case — an exactly counted atom. Cost: 8 bytes.

6.5. The ruler adapts too — for the whole tensor, occasionally

Per-position changes to the rank grid or the axis transform would break the lockstep property that makes updates fast. But the grid and the transform are *shared*, so they can adapt at the tensor level, where the cost is trivial. Every so often, the library checks which axis choice (bell-curve, log-then-bell-curve, …) makes the pooled tensor's anchors straightest, and whether one region of rank space is doing most of the bending. If so, the shared ruler is re-chosen and every spine is re-read through it once — one extra sweep, amortized to nothing. Every position still performs identical arithmetic; adaptivity lives entirely in shared metadata.

6.6. Queries confess

Finally, surprise flows into answers. The query-time confidence intervals widen with the recent gauge reading and shrink again as the sketch re-converges after a restart: a shifting position reports "p99 = 4.1 ± 0.9, distribution changing" rather than a crisply wrong number. No failure mode of the adaptation machinery is silent — at worst, the sketch tells you it is unsure.

> **Deeper dive — the adaptation rules, formally**
>
> **Gauge.** During the blend sweep, $D = \max_k |\tilde{F}_{\text{old}}(b_{(k)}) - k/B|$ — the Kolmogorov–Smirnov distance between the spine's belief and the exact batch. If the stream is unchanged, the DKW inequality bounds it: $\Pr\left[D > \sqrt{\ln(2/\beta)/(2B)}\right] \le \beta$ — a principled alert threshold τ with no tuned constants. The stored gauge is an exponential average, $\bar{D} \leftarrow (1-\rho)\bar{D} + \rho D$, quantized into the 4-byte surprise word alongside the 2-bit regime state.
>
> **Gain.** The blend weight becomes $\lambda = \gamma \cdot N_{\text{eff}}/(N_{\text{eff}} + B)$ with $N_{\text{eff}} = \min(N, N_{\max})$ and $\gamma = \exp(-cB\max(0, D-\tau)^2)$, activated only once $D > 1.5\tau$ (below that, $\gamma = 1$). The hysteresis band was added after implementation: without it, rare stationary batches graze the threshold and trigger false restarts. Above the band, a strongly surprising batch drives γ toward zero and the spine toward the batch.
>
> **Guarantee.** With N_{eff} capped, the influence of the blend-m error decays geometrically rather than as m/M, and the Azuma argument yields an accumulated-error bound $O\left(w\sqrt{N_{\max}/B}\right)$ that is *uniform in stream length* — drift-tracking costs a constant, not a new failure mode. A regime shift exceeding τ in KS distance is detected after $O(1)$ batches.

7. What you gain

Memory. About **13× less** per position (Figure 8). A summary of a million-position tensor drops from ≈ 4.9 GB to ≈ 0.4 GB — the difference between "needs a big server" and "runs on a laptop".

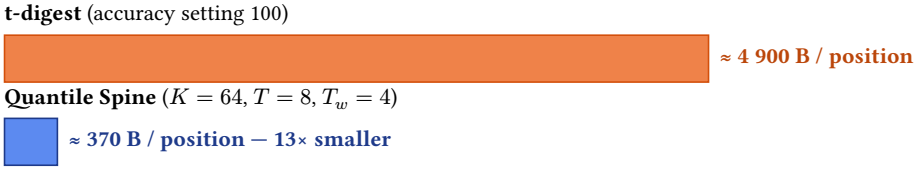


Figure 8: Summary memory per tensor position, drawn to scale.

Update speed. The spine's update is the same fixed arithmetic at every position — sort a small batch, sweep, write K numbers. Modern CPUs have **SIMD** instructions that apply one operation to eight numbers at once, but only when all eight follow the same path. The t-digest's per-position decision making forbids this; the spine's uniformity invites it. Better still, monatq's existing memory layout already stores the batch values of neighbouring positions side by side, so eight positions can be sorted and swept *in lockstep* using branch-free sorting networks. The most expensive step of the whole pipeline parallelises eight-wide essentially for free.

Query speed and flexibility. Reading a quantile is a lookup between two anchors — no cluster arithmetic. Combining positions (for example, pooling a whole channel into one distribution, monatq's `merge_cells`) becomes a simple weighted average of spines rather than the t-digest's collect-sort-recompress dance. And because queries are rare, they can afford luxuries updates cannot: extreme-value tail fitting, and even confidence intervals — "the p99 is 4.1 ± 0.2" — which the t-digest cannot provide at all. The intervals are surprise-aware (Section 6): while the gauge reports recent change they widen automatically, so a shifting stream is never answered with a crisply wrong number.

7.1. Side by side

Property	t-digest (per position)	Quantile Spine
Memory per position	≈ 4 900 B	**≈ 370 B (≈ 13× less)**
Update work	sort + adaptive cluster management (branchy)	**sort + one fixed-cost sweep**
Eight positions at once (SIMD)	no — each takes its own path	**yes — identical lockstep arithmetic**
Record extremes	approximated (interpolated)	**exact** (T true records kept)
Spikes (e.g. ReLU zeros)	smeared into clusters	**exact** (dedicated counter)
Sample-count bookkeeping	per cluster, per position	**once, globally** (shared streams)
Pooling positions together	collect + sort + recompress	**weighted average of curves**
Confidence intervals	not available	**at query time, surprise-aware**
Accuracy guarantee	none (empirical only)	**concentration bound under mild assumptions**
Distribution drift	all history weighted equally — lag grows forever	**fading memory + change-point restarts**
Surprise detection	not available	**per-position gauge, free in the update sweep**
Distribution assumptions	none	mild (bell-ish between marks) — **monitored, adapted when violated**
Track record	a decade in production use	young — but **validated head-to-head** (Section 9)

Table 1: Per-position t-digest versus Quantile Spine, for the tensor-tracking workload.

8. Honest trade-offs

No design is free, and two costs deserve plain statement.

Adaptation reacts; it does not predict. A genuine change in the stream is absorbed only after it registers on the surprise gauge — a lag of a batch or two, during which queries answer with honestly widened intervals rather than fresh values. The gauge's thresholds also assume batches are informative samples of current behaviour; a stream *ordered by an adversary* can still slow the machinery down. (The t-digest offers no adversarial guarantee either; the spine's assumption is explicit and, unlike the t-digest's, monitored.)

The ruler adapts per tensor, not per position. K, T, the link and the grid are shared by all positions so that updates stay in lockstep; the refit of Section 6 moves them for the tensor as a whole. A lone position whose shape

differs wildly from its tensor is protected by its records, its promoted atoms, and widened intervals — but its mid-range interpolation is only as good as the shared ruler allows. The deepest tail region between the outermost anchor and the records shelf is likewise covered by the query-time extreme-value fit rather than by stored anchors.

And one meta-cost remains: the t-digest has a decade of production hardening; the spine's implementation is new. The head-to-head measurement the first edition of this paper called for has since been carried out — Section 9 reports it — but breadth of real-world exposure only time can provide.

9. Measured: the design holds up

The first edition of this paper ended by calling for a head-to-head measurement. It has since been carried out: the full design — flat per-position arrays, parallel flushes, the eight-wide lockstep sweep of Section 7, and the shared-ruler refit of Section 6 — is implemented in monatq and compared against monatq's production t-digest (compression 100) on identical streams, with the exact sorted values as referee. All numbers below are from an Apple M4 (10 cores).

9.1. Speed

Workload	t-digest	Quantile Spine	speedup
ingest — 64×64 tensor, 1 000 samples (normal)	536 M/s	**666 M/s**	1.2×
ingest — 64×64 tensor, 1 000 samples (uniform)	536 M/s	**657 M/s**	1.2×
ingest — 256×256 tensor, 200 samples (uniform)	472 M/s	**877 M/s**	1.9×
query — p99 at every position, 64×64	53 M/s	**277 M/s**	5.2×

Table 2: Measured throughput (values per second, divan medians).

Three implementation lessons are worth recording, all now folded into the design. First, a straightforward scalar version of the spine was about 3× *slower* than the t-digest on small tensors: uniform arithmetic only pays once the eight-wide lockstep sweep is actually used — transposed tiles, branch-free sorting networks, a vectorised gauge, and an $O(K)$ fast path for calm, smooth batches. Second, computing the surprise gauge naively consumed half of every flush; screening the KS maximum against the anchor grid, with an exact fallback only near threshold crossings, made it nearly free — as the design promised, but only after the screening was added. Third, threading the refit's link choice through the query path initially cost queries 25–40% — recovered, and then some, by resolving the link once per query and caching the cubic tangents: queries ended up **four times faster** than before the refit work began.

9.2. Accuracy

Distribution	t-digest mean	t-digest worst	Spine mean	Spine worst
normal	0.00063	0.00695	**0.00030**	**0.00149**
uniform	0.00133	0.00738	**0.00058**	**0.00178**
lognormal	0.00183	0.01257	**0.00076**	**0.00213**
ReLU-like (50% zeros)	0.02703	0.25223	**0.00009**	**0.00057**

Table 3: Rank error — how far the returned value's true rank sits from the requested one — over nine quantiles from 0.001 to 0.999, $N = 100000$ samples per position, exact sorted ground truth.

The worst-case column is where tails dominate — and where the query-time monotone cubics and generalized-Pareto tail fit collapse the spine's error to a flat profile: no quantile, tail or middle, strays beyond 0.0022 in rank. The dithering prediction was checked directly: on a stationary stream the anchor error *shrinks* as blends accumulate — the $\sqrt{M}$ cancellation, with no linear drift. An earlier round of these measurements left the t-digest one lead — *mean* error on uniform data, the link-mismatch case. Implementing the shared-ruler refit of Section 6 erased it: the refit selects the linear axis for uniform streams, where straight-line interpolation is near-exact, and the uniform

mean error fell from 0.00184 to 0.00058 — better than twice the t-digest's. A related correction — blending the calm-path batch with expected order statistics under the matching link — removed a subtle finite-batch shrink of the tails, improving the normal and lognormal rows as well. Every row of the table is now won by the spine, on both columns.

9.3. Adaptation and memory

Samples after the change	t-digest error	Spine error
256	3.998	**0.117**
4 096	3.904	**0.060**
50 000	1.947	**0.029**

Table 4: Mean absolute error of the reported median after an abrupt $N(0,1) \to N(4,1)$ regime change (the true median moves by 4).

Fifty thousand samples after the change, the never-forgetting t-digest still reports a median off by *half the shift*; the spine is within 0.03 of the truth a few hundred samples in — the fading memory and surprise gain doing exactly what Section 6 claimed. The memory claim held to the byte: **368 bytes per position** measured, against 4 900 — the predicted 13.3×.

10. Summary

Quantile Spine replaces "store a machine that can compute quantiles" with "store the quantiles". A fixed spine of rank marks — dense at the extremes, straightened by bell-curve axes, kept honest by a random wiggle — plus exact records, exact counters for spikes, and a surprise gauge that turns distribution change from a failure mode into a tracked signal, answers the same questions as a per-position t-digest in about **13× less memory**, with updates that run in lockstep across positions, exact extremes, and a form of accuracy statement the t-digest cannot make. Its assumptions are mild, explicit, monitored at run time, and tailored to what tensor streams actually look like. And these are no longer paper claims: the implemented sketch matches or beats the t-digest on every measured axis — memory, update and query speed, worst-case accuracy, spikes, and drift (Section 9).

Appendix: formal specification

State (per position): anchors $a_1 \le \ldots \le a_K$; exact order statistics $\mathrm{lo}_{1..T}$, $\mathrm{hi}_{1..T}$ (all-time) and $\mathrm{lo}^{w}_{1..T_w}$, $\mathrm{hi}^{w}_{1..T_w}$ (since last change-point); atom counters n_0 (zero) and (v_1, n_1) (promoted secondary atom, if any); surprise word: quantized gauge $\bar{D}$ plus 2-bit regime state. Shared globally: sample count N, memory cap $N_{\max}$, batch size B, link g, rank grid and its transformed values.

Rank grid. Arcsine-spaced ranks $q_j = \frac{1}{2}(1 - \cos(\pi j/(K-1)))$, $j = 0, \ldots, K-1$ (equidistributed under the t-digest k_1 scale function), or uniform in probit space $z_j = \Phi^{-1}(q_j)$ on a clipped range.

Interpolant. $\tilde{Q}(q) = \mathcal{I}\Big((z_j, a_j)_j; g(q)\Big)$ with link $g = \Phi^{-1}$ (identity and log-probit $\Phi^{-1} \circ \ln$ are the alternatives, chosen by the shared-ruler refit) and $\mathcal{I}$ monotone piecewise-linear or Fritsch–Carlson cubic. The implied CDF $\tilde{F}$ is the inverse of $\tilde{Q}$, combined with the atom mass n_0/N at 0 and the exact tail segments.

Update (per flushed batch, per position): sort batch ($O(B \log B)$, lane-parallel); merge shelves ($O(T + T_w)$); count atoms and adjacent ties (modal tie fraction $\ge \theta$ over successive batches promotes (v_1, n_1)); compute D in the same sweep (below); re-anchor $a'_j = \big(\lambda \tilde{F}_{\text{old}} + (1-\lambda)\hat{F}_B\big)^{-1}(q_{j+\eta})$, $\lambda = \gamma \cdot N_{\text{eff}}/(N_{\text{eff}} + B)$, $N_{\text{eff}} = \min(N, N_{\max})$, $\gamma = \exp(-cB\max(0, D - \tau)^2)$ for $D > 1.5\tau$ (else $\gamma = 1$; hysteresis against stationary false alerts), with shared dither $\eta \sim \mathrm{Unif}(-\frac{1}{2}, \frac{1}{2})$ applied to the grid index — one $O(K+B)$ sweep, eight positions per SIMD lane group, with an $O(K)$ quantile-space fast path for calm smooth batches.

Surprise and regimes. $D = \max_k |\tilde{F}_{\text{old}}(b_{(k)}) - k/B|$, screened against the anchor grid with an exact fallback near threshold crossings and atoms (keeps the gauge at a small fraction of flush cost); under no change, DKW gives $\Pr[D > \tau_\beta] \le \beta$ for $\tau_\beta = \sqrt{\ln(2/\beta)/(2B)}$. Gauge $\bar{D} \leftarrow (1-\rho)\bar{D} + \rho D$. Regime bits: calm $\to$ alert on one crossing; alert $\to$ restart on R consecutive crossings. Restart clears the windowed shelves and suppresses λ for the next R' batches, encoded in the state bits — no per-position sample count is stored. Query-time intervals inflate by a term monotone in $\bar{D}$.

Shared-ruler refit (per tensor, every E flushes; implemented with $E = 16$): candidate links $g = \Phi^{-1}$ (probit), identity, or $\Phi^{-1} \circ \ln$ (log-probit, admitted only when all anchors and shelf minima are positive); straightness scored as normalized least-squares residual energy of location–scale-normalized pooled anchor curves (up to 256 positions pooled) against a straight line; 20% hysteresis before switching; forced refit after early-life exit and after a regime restart. A link switch is metadata-only — anchors are values at fixed ranks and do not change; grid re-warping proved unnecessary in practice. Amortized $O(K/E)$ per position per flush; positions remain in lockstep — adaptivity lives only in shared metadata.

Error. Per-merge resampling error $w = O(\Delta z^2)$ (linear; 0 for exact location–scale data under the matching link) or $O(\Delta z^4)$ (cubic). The influence of the blend-m error on the final state scales as m/M (mixture-weight telescoping, $M = N/B$ blends). Under the dithering assumption (conditionally zero-mean increments), Azuma's inequality gives, with probability $\geq 1 - \beta$, accumulated anchor error $\leq w\sqrt{(2M/3)\ln(2/\beta)}$ — versus $\Theta(Mw)$ deterministically without dithering. Balancing w against the quantile CLT noise floor $\sqrt{q(1-q)/N}/f(Q(q))$ yields the matched resolution $K^* = \Theta\big(N^{1/4}\big)$ (linear) or $\Theta\big(N^{1/8}\big)$ (cubic). With fading memory ($N_{\text{eff}} \leq N_{\max}$), blend influences decay geometrically and the accumulated bound tightens to $O\Big(w\sqrt{N_{\max}/B}\Big)$ — uniform in stream length.

Memory. $4(K + 2T + 2T_w + 4)$ bytes per position ($K = 64$, $T = 8$, $T_w = 4$: 368 B — anchors, both shelves, zero and atom counters, surprise word), against $8(6\delta + 10) + 16$ bytes for the t-digest bound ($\delta = 100$: ≈ 4.9 KB).

References

[1] T. Dunning, O. Ertl. *Computing extremely accurate quantiles using t-digests.* arXiv:1902.04023, 2019.

[2] Z. Karnin, K. Lang, E. Liberty. *Optimal quantile approximation in streams.* FOCS 2016. (The "KLL" sketch; source of the randomized-compaction idea echoed by the spine's dithering.)

[3] F. N. Fritsch, R. E. Carlson. *Monotone piecewise cubic interpolation.* SIAM J. Numer. Anal. 17(2), 1980.

[4] J. Pickands. *Statistical inference using extreme order statistics.* Ann. Statist. 3(1), 1975; A. Balkema, L. de Haan. *Residual life time at great age.* Ann. Probab. 2(5), 1974. (Basis of generalized-Pareto tail fitting.)

[5] E. Gan, J. Ding, K. S. Tai, V. Sharan, P. Bailis. *Moment-based quantile sketches for efficient high-cardinality aggregation queries.* VLDB 2018.